

# **SIMATS ENGINEERING**

## **ASSESSMENT TOOL 1**

**COURSE CODE: ITA 06**

**COURSE NAME: MACHINE LEARNING**

### **COURSE OUTCOME COVERED**

***CO3: Bayes Theorem – Concept Learning – Maximum Likelihood – Minimum Description Length Principle – Bayes Optimal Classifier – Gibbs Algorithm – Naïve Bayes Classifier – Bayesian Belief Network – EM Algorithm – Probability Learning – Sample Complexity – Finite and Infinite Hypothesis Spaces – Mistake Bound Model, (BL 6)***

*Assessment Tool Weightage: 40%*

**QUESTIONS: 15**

**Total Marks:25**

Annexure A

### **Experiments**

#### ***Code of Conduct:***

*I Pravin R / 192421292 certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.*

*I understand that any violation of academic integrity rules will result in disciplinary action.*

***Signature of the student:***

## Annexure A

- 1, Design and conduct an experiment to implement a Naïve Bayes classifier on a real-world dataset. Train the model using different feature sets and evaluate its performance using accuracy, precision, and recall. Analyze how assumptions of feature independence affect the results. Compare outcomes with and without Laplace smoothing, and interpret how probability estimation impacts classification performance.
2. Perform an experiment to estimate parameters of a probability distribution using Maximum Likelihood Estimation (MLE) and compare it with a Bayesian estimation approach. Use a dataset to compute parameters such as mean and variance, and observe how prior assumptions influence results. Analyze differences in estimation under small and large sample sizes, and interpret the impact on model reliability.
3. Implement the Expectation-Maximization (EM) algorithm for a mixture model and conduct experiments to study its convergence behavior. Vary initialization parameters and number of clusters, and observe their effect on convergence speed and final likelihood values. Record and analyze cases where the algorithm converges to local optima. Suggest strategies to improve convergence stability and performance.
4. Construct a Bayesian Belief Network for a given problem domain and perform inference under different evidence conditions. Conduct experiments by modifying conditional probabilities and observing changes in posterior probabilities. Analyze how dependencies between variables influence the results. Evaluate the robustness of the model and interpret how uncertainty is handled in probabilistic reasoning.
5. Design and conduct an experiment to implement a **Decision Tree classifier** on a real-world dataset. Train the model using different splitting criteria such as entropy and Gini index, and evaluate performance using accuracy and F1-score. Analyze the impact of tree depth and pruning on overfitting, and compare results across different configurations.

## RUBRICS FOR CALCULATION

| Criteria                          | Weightage | Level 4 – Exemplary (5 Marks)                                | Level 3 – Proficient (4 Marks)         | Level 2 – Developing (2–3 Marks)             | Level 1 – Beginning (0–1 Mark)  |
|-----------------------------------|-----------|--------------------------------------------------------------|----------------------------------------|----------------------------------------------|---------------------------------|
| <b>Conceptual Understanding</b>   | <b>30</b> | Demonstrates clear and complete understanding of the concept | Good understanding with minor gaps     | Basic understanding with some misconceptions | Poor or incorrect understanding |
| <b>Accuracy of Answer</b>         | <b>25</b> | Completely correct answer with no errors                     | Mostly correct with minor errors       | Partially correct with noticeable errors     | Incorrect or irrelevant answer  |
| <b>Application of Knowledge</b>   | <b>15</b> | Correctly applies concepts to solve the question/problem     | Applies concepts with minor mistakes   | Limited or partially correct application     | No or incorrect application     |
| <b>Completeness of Response</b>   | <b>15</b> | Covers all required parts of the question                    | Covers most parts with minor omissions | Covers only some parts                       | Incomplete or missing response  |
| <b>Clarity &amp; Presentation</b> | <b>15</b> | Clear, concise, and well-organized answer                    | Understandable with minor issues       | Somewhat unclear or poorly structured        | Difficult to understand         |

## EXPERIMENT 1 — Naïve Bayes Classifier

### Dataset

#### Pima Indians Diabetes Database

|       | Pregna<br>ncies | Glucos<br>e    | BloodPr<br>essure | SkinThi<br>ckness | Insulin        | BMI            | Diabete<br>sPedigr<br>eeFunc<br>tion | Age            | Outcom<br>e    |
|-------|-----------------|----------------|-------------------|-------------------|----------------|----------------|--------------------------------------|----------------|----------------|
| count | 768.000<br>000  | 768.000<br>000 | 768.000<br>000    | 768.000<br>000    | 768.000<br>000 | 768.000<br>000 | 768.000<br>000                       | 768.000<br>000 | 768.000<br>000 |
| mean  | 3.84505<br>2    | 120.894<br>531 | 69.1054<br>69     | 20.5364<br>58     | 79.7994<br>79  | 31.9925<br>78  | 0.47187<br>6                         | 33.2408<br>85  | 0.34895<br>8   |
| std   | 3.36957<br>8    | 31.9726<br>18  | 19.3558<br>07     | 15.9522<br>18     | 115.244<br>002 | 7.88416<br>0   | 0.33132<br>9                         | 11.7602<br>32  | 0.47695<br>1   |
| min   | 0.00000<br>0    | 0.00000<br>0   | 0.00000<br>0      | 0.00000<br>0      | 0.00000<br>0   | 0.00000<br>0   | 0.07800<br>0                         | 21.0000<br>00  | 0.00000<br>0   |
| 25%   | 1.00000<br>0    | 99.0000<br>00  | 62.0000<br>00     | 0.00000<br>0      | 0.00000<br>0   | 27.3000<br>00  | 0.24375<br>0                         | 24.0000<br>00  | 0.00000<br>0   |
| 50%   | 3.00000<br>0    | 117.000<br>000 | 72.0000<br>00     | 23.0000<br>00     | 30.5000<br>00  | 32.0000<br>00  | 0.37250<br>0                         | 29.0000<br>00  | 0.00000<br>0   |
| 75%   | 6.00000<br>0    | 140.250<br>000 | 80.0000<br>00     | 32.0000<br>00     | 127.250<br>000 | 36.6000<br>00  | 0.62625<br>0                         | 41.0000<br>00  | 1.00000<br>0   |
| max   | 17.0000<br>00   | 199.000<br>000 | 122.000<br>000    | 99.0000<br>00     | 846.000<br>000 | 67.1000<br>00  | 2.42000<br>0                         | 81.0000<br>00  | 1.00000<br>0   |

## Code

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.model_selection import train_test_split
from sklearn.naive_bayes import GaussianNB, CategoricalNB
from sklearn.preprocessing import KBinsDiscretizer
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score,
confusion_matrix, classification_report

url =
"https://raw.githubusercontent.com/npradaschnor/Pima-Indians-Diabetes-Dataset/master/diabetes.csv"
df = pd.read_csv(url)

print(df.head())
print("\nShape:", df.shape)
print("\nColumns:", df.columns.tolist())
print("\nMissing Values:\n", df.isnull().sum())
print("\nStatistics:\n", df.describe())

X = df.drop("Outcome", axis=1)
y = df["Outcome"]

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.20, random_state=42, stratify=y
)

model = GaussianNB()
model.fit(X_train, y_train)
y_pred = model.predict(X_test)

print("\nNaive Bayes Results")
print("Accuracy :", accuracy_score(y_test, y_pred))
print("Precision:", precision_score(y_test, y_pred))
print("Recall  :", recall_score(y_test, y_pred))
print("F1 Score :", f1_score(y_test, y_pred))
print("\nClassification Report:\n")
print(classification_report(y_test, y_pred))

cm = confusion_matrix(y_test, y_pred)

plt.figure(figsize=(6, 5))
sns.heatmap(
    cm,
    annot=True,
```

```

    fmt="d",
    cmap="Blues",
    xticklabels=["No Diabetes", "Diabetes"],
    yticklabels=["No Diabetes", "Diabetes"]
)
plt.xlabel("Predicted")
plt.ylabel("Actual")
plt.title("Naive Bayes Confusion Matrix")
plt.show()

feature_sets = {
    "All Features": [
        "Pregnancies", "Glucose", "BloodPressure",
        "SkinThickness", "Insulin", "BMI",
        "DiabetesPedigreeFunction", "Age"
    ],
    "Glucose + BMI": [
        "Glucose", "BMI"
    ],
    "Glucose + BMI + Age": [
        "Glucose", "BMI", "Age"
    ],
    "Glucose + BloodPressure + BMI + Age": [
        "Glucose", "BloodPressure", "BMI", "Age"
    ],
    "Glucose + Insulin + BMI": [
        "Glucose", "Insulin", "BMI"
    ]
}

results = []

for name, features in feature_sets.items():
    X_selected = df[features]

    X_train, X_test, y_train, y_test = train_test_split(
        X_selected,
        y,
        test_size=0.20,
        random_state=42,
        stratify=y
    )

    model = GaussianNB()
    model.fit(X_train, y_train)
    predictions = model.predict(X_test)

    results.append({
        "Feature Set": name,
        "Features": len(features),

```

```

        "Accuracy": accuracy_score(y_test, predictions),
        "Precision": precision_score(y_test, predictions),
        "Recall": recall_score(y_test, predictions),
        "F1 Score": f1_score(y_test, predictions)
    })

results_df = pd.DataFrame(results)

print("\nFeature Set Comparison")
display(results_df)

results_melted = results_df.melt(
    id_vars="Feature Set",
    value_vars=["Accuracy", "Precision", "Recall", "F1 Score"],
    var_name="Metric",
    value_name="Score"
)

plt.figure(figsize=(12, 6))
sns.barplot(
    data=results_melted,
    x="Feature Set",
    y="Score",
    hue="Metric"
)
plt.xticks(rotation=25, ha="right")
plt.ylim(0, 1)
plt.title("Naive Bayes Performance for Different Feature Sets")
plt.tight_layout()
plt.show()

plt.figure(figsize=(10, 7))
sns.heatmap(
    df.drop("Outcome", axis=1).corr(),
    annot=True,
    fmt=".2f",
    cmap="coolwarm"
)
plt.title("Feature Correlation Matrix")
plt.show()

X = df.drop("Outcome", axis=1)

discretizer = KBinsDiscretizer(
    n_bins=5,
    encode="ordinal",
    strategy="quantile"
)

X_discrete = discretizer.fit_transform(X)

```

```

X_discrete = pd.DataFrame(
    X_discrete,
    columns=X.columns
).astype(int)

X_train, X_test, y_train, y_test = train_test_split(
    X_discrete,
    y,
    test_size=0.20,
    random_state=42,
    stratify=y
)

alpha_values = [0, 0.1, 0.5, 1, 5, 10]
smoothing_results = []

for alpha in alpha_values:
    model = CategoricalNB(alpha=alpha)
    model.fit(X_train, y_train)
    predictions = model.predict(X_test)

    smoothing_results.append({
        "Alpha": alpha,
        "Accuracy": accuracy_score(y_test, predictions),
        "Precision": precision_score(y_test, predictions),
        "Recall": recall_score(y_test, predictions),
        "F1 Score": f1_score(y_test, predictions)
    })

smoothing_df = pd.DataFrame(smoothing_results)

print("\nLaplace Smoothing Comparison")
display(smoothing_df)

plt.figure(figsize=(10, 6))

plt.plot(
    smoothing_df["Alpha"],
    smoothing_df["Accuracy"],
    marker="o",
    label="Accuracy"
)

plt.plot(
    smoothing_df["Alpha"],
    smoothing_df["Precision"],
    marker="s",
    label="Precision"
)

```



```
plt.plot(
    smoothing_df["Alpha"],
    smoothing_df["Recall"],
    marker="^",
    label="Recall"
)

plt.plot(
    smoothing_df["Alpha"],
    smoothing_df["F1 Score"],
    marker="D",
    label="F1 Score"
)

plt.xlabel("Alpha")
plt.ylabel("Score")
plt.title("Effect of Laplace Smoothing")
plt.legend()
plt.grid(True)
plt.show()
```

## Output

```
...
Pregnancies  Glucose  BloodPressure  SkinThickness  Insulin  BMI  \
0           6       148           72           35         0   33.6
1           1        85           66           29         0   26.6
2           8       183           64            0         0   23.3
3           1        89           66           23         94   28.1
4           0       137           40           35        168   43.1

DiabetesPedigreeFunction  Age  Outcome
0           0.627    50         1
1           0.351    31         0
2           0.672    32         1
3           0.167    21         0
4           2.288    33         1

Shape: (768, 9)

Columns: ['Pregnancies', 'Glucose', 'BloodPressure', 'SkinThickness', 'Insulin', 'BMI', 'DiabetesPedigreeFunction', 'Age', 'Outcome']

Missing Values:
Pregnancies      0
Glucose          0
BloodPressure    0
SkinThickness    0
Insulin          0
BMI              0
DiabetesPedigreeFunction  0
Age              0
Outcome          0
dtype: int64

Statistics:
Pregnancies  Glucose  BloodPressure  SkinThickness  Insulin  \
count  768.000000  768.000000  768.000000  768.000000  768.000000
mean    3.845052  120.894531   69.105469   20.536458   79.799479
std     3.369578   31.972618   19.355807   15.952218  115.244002
min     0.000000   0.000000   0.000000   0.000000   0.000000
25%     1.000000   99.000000   62.000000   0.000000   0.000000
50%     3.000000  117.000000   72.000000   23.000000  30.500000
75%     6.000000  140.250000   80.000000   32.000000  127.250000
max    17.000000  199.000000  122.000000   99.000000  846.000000
```

```

...   count    BMI  DiabetesPedigreeFunction    Age    Outcome
     mean    31.992578      0.471876    33.240885    0.348958
     std     7.884160      0.331329    11.760232    0.476951
     min     0.000000      0.078000    21.000000    0.000000
    25%     27.300000      0.243750    24.000000    0.000000
    50%     32.000000      0.372500    29.000000    0.000000
    75%     36.600000      0.626250    41.000000    1.000000
    max     67.100000      2.420000    81.000000    1.000000

```

#### Naive Bayes Results

Accuracy : 0.7077922077922078

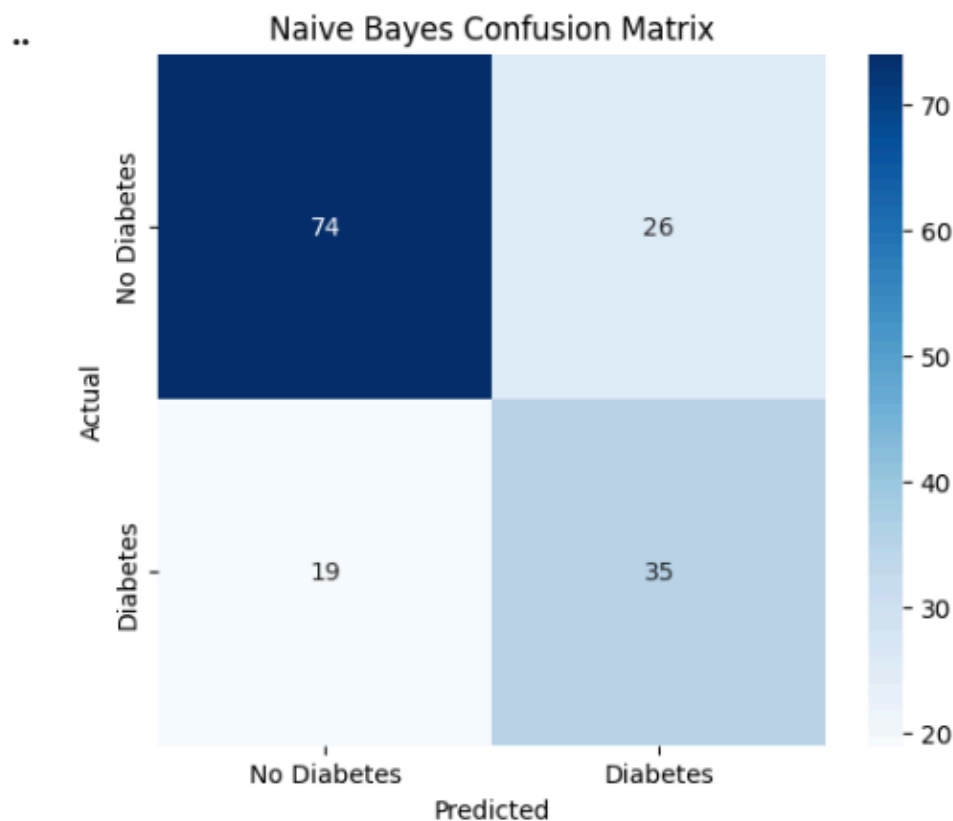
Precision: 0.5737704918032787

Recall : 0.6481481481481481

F1 Score : 0.6086956521739131

#### Classification Report:

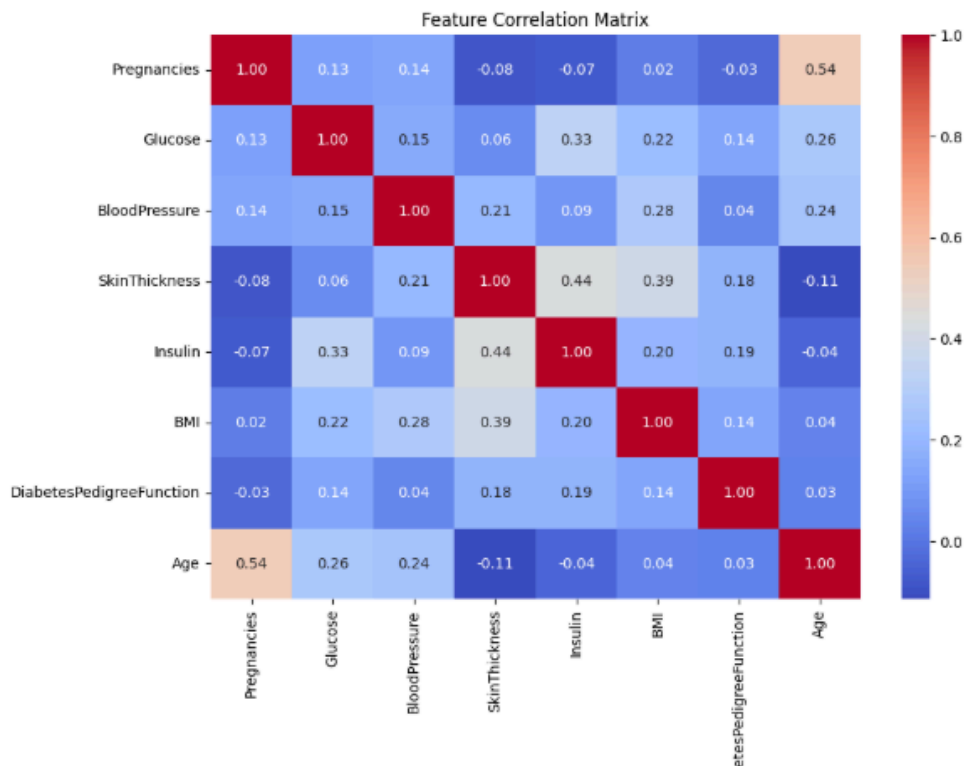
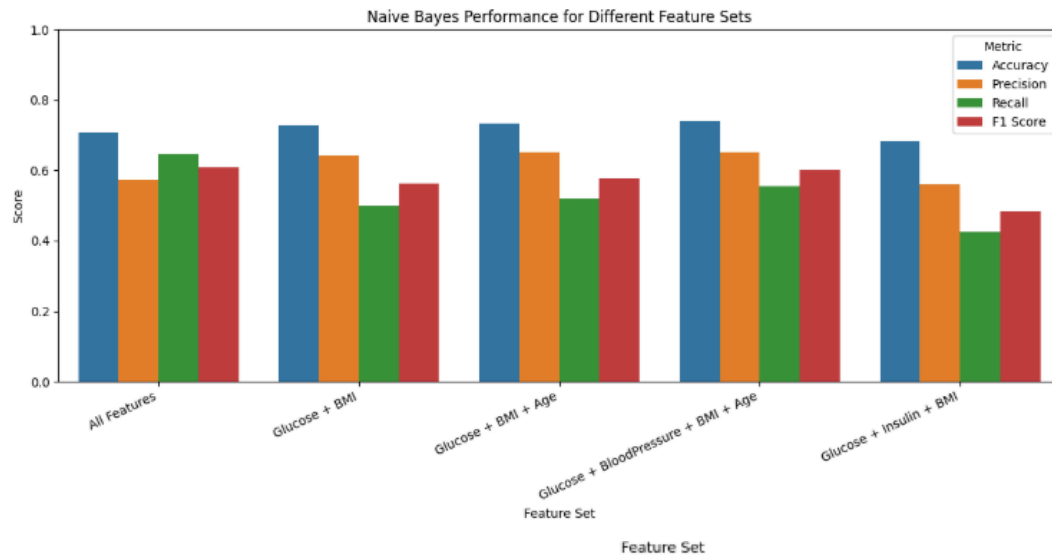
|              | precision | recall | f1-score | support |
|--------------|-----------|--------|----------|---------|
| 0            | 0.80      | 0.74   | 0.77     | 100     |
| 1            | 0.57      | 0.65   | 0.61     | 54      |
| accuracy     |           |        | 0.71     | 154     |
| macro avg    | 0.68      | 0.69   | 0.69     | 154     |
| weighted avg | 0.72      | 0.71   | 0.71     | 154     |

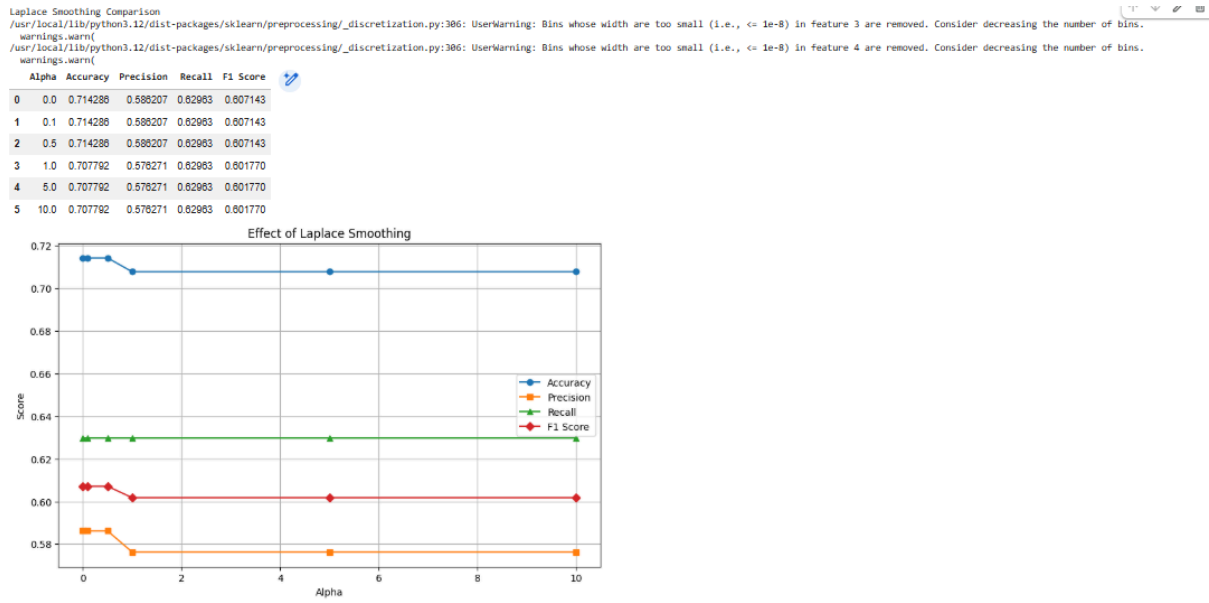


...

### Feature Set Comparison

|   | Feature Set                         | Features | Accuracy | Precision | Recall   | F1 Score |
|---|-------------------------------------|----------|----------|-----------|----------|----------|
| 0 | All Features                        | 8        | 0.707792 | 0.573770  | 0.648148 | 0.608696 |
| 1 | Glucose + BMI                       | 2        | 0.727273 | 0.642857  | 0.500000 | 0.562500 |
| 2 | Glucose + BMI + Age                 | 3        | 0.733766 | 0.651163  | 0.518519 | 0.577320 |
| 3 | Glucose + BloodPressure + BMI + Age | 4        | 0.740260 | 0.652174  | 0.555556 | 0.600000 |
| 4 | Glucose + Insulin + BMI             | 3        | 0.681818 | 0.560976  | 0.425926 | 0.484211 |





## EXPERIMENT 2 — MLE vs Bayesian Estimation

### Wine Quality dataset.

| fixed acidity | volatile acidity | citric acid | residual sugar | chlorides | free sulfur dioxide | total sulfur dioxide | density |
|---------------|------------------|-------------|----------------|-----------|---------------------|----------------------|---------|
| 7.4           | 0.7              | 0           | 1.9            | 0.076     | 11                  | 34                   | 0.9978  |
| 7.8           | 0.88             | 0           | 2.6            | 0.098     | 25                  | 67                   | 0.9968  |
| 7.8           | 0.76             | 0.04        | 2.3            | 0.092     | 15                  | 54                   | 0.997   |
| 11.2          | 0.28             | 0.56        | 1.9            | 0.075     | 17                  | 60                   | 0.998   |
| 7.4           | 0.7              | 0           | 1.9            | 0.076     | 11                  | 34                   | 0.9978  |
| 7.4           | 0.66             | 0           | 1.8            | 0.075     | 13                  | 40                   | 0.9978  |
| 7.9           | 0.6              | 0.06        | 1.6            | 0.069     | 15                  | 59                   | 0.9964  |
| 7.3           | 0.65             | 0           | 1.2            | 0.065     | 15                  | 21                   | 0.9946  |
| 7.8           | 0.58             | 0.02        | 2              | 0.073     | 9                   | 18                   | 0.9968  |
| 6.7           | 0.58             | 0.08        | 1.8            | 0.097     | 15                  | 65                   | 0.9959  |

# Code

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

data = {
    "fixed acidity": [7.4, 7.8, 7.8, 11.2, 7.4, 7.4, 7.9, 7.3, 7.8, 6.7],
    "volatile acidity": [0.7, 0.88, 0.76, 0.28, 0.7, 0.66, 0.6, 0.65, 0.58, 0.58],
    "citric acid": [0, 0, 0.04, 0.56, 0, 0, 0.06, 0, 0.02, 0.08],
    "residual sugar": [1.9, 2.6, 2.3, 1.9, 1.9, 1.8, 1.6, 1.2, 2, 1.8],
    "chlorides": [0.076, 0.098, 0.092, 0.075, 0.076, 0.075, 0.069, 0.065, 0.073, 0.097],
    "free sulfur dioxide": [11, 25, 15, 17, 11, 13, 15, 15, 9, 15],
    "total sulfur dioxide": [34, 67, 54, 60, 34, 40, 59, 21, 18, 65],
    "density": [0.9978, 0.9968, 0.997, 0.998, 0.9978, 0.9978, 0.9964, 0.9946, 0.9968, 0.9959]
}

df = pd.DataFrame(data)

print("Dataset:")
display(df)

print("Shape:", df.shape)

print("\nStatistics:")
display(df.describe())

data = df["fixed acidity"].values

n = len(data)

mle_mean = np.mean(data)

mle_variance = np.mean(
    (data - mle_mean) ** 2
)

sample_mean = np.mean(data)

sample_variance = np.var(
    data,
    ddof=1
)

print("\nMLE Estimation")
print("MLE Mean:", mle_mean)
print("MLE Variance:", mle_variance)

print("\nSample Estimation")
```

```

print("Sample Mean:", sample_mean)
print("Sample Variance:", sample_variance)

def bayesian_estimation(
    data,
    prior_mean,
    prior_variance
):
    n = len(data)

    sample_mean = np.mean(data)

    sample_variance = np.var(
        data,
        ddof=1
    )

    posterior_variance = 1 / (
        (1 / prior_variance) +
        (n / sample_variance)
    )

    posterior_mean = posterior_variance * (
        (prior_mean / prior_variance) +
        (n * sample_mean / sample_variance)
    )

    return posterior_mean, posterior_variance

priors = [
    (5, 1),
    (7, 1),
    (8, 1),
    (10, 1),
    (12, 1)
]

prior_results = []

for prior_mean, prior_variance in priors:

    posterior_mean, posterior_variance = bayesian_estimation(
        data,
        prior_mean,
        prior_variance
    )

    prior_results.append({
        "Prior Mean": prior_mean,
        "Prior Variance": prior_variance,

```

```

        "Posterior Mean": posterior_mean,
        "Posterior Variance": posterior_variance
    })

prior_df = pd.DataFrame(prior_results)

print("\nBayesian Estimation with Different Priors")
display(prior_df)

sample_sizes = [2, 3, 4, 5, 6, 7, 8, 9, 10]

sample_results = []

for size in sample_sizes:

    sample = data[:size]

    mle = np.mean(sample)

    bayesian_mean, bayesian_variance = bayesian_estimation(
        sample,
        prior_mean=8,
        prior_variance=1
    )

    sample_results.append({
        "Sample Size": size,
        "MLE Mean": mle,
        "Bayesian Mean": bayesian_mean,
        "MLE Variance": np.var(sample, ddof=0),
        "Bayesian Variance": bayesian_variance
    })

sample_df = pd.DataFrame(sample_results)

print("\nSmall vs Large Sample Size")
display(sample_df)

plt.figure(figsize=(10, 6))

plt.plot(
    sample_df["Sample Size"],
    sample_df["MLE Mean"],
    marker="o",
    label="MLE Mean"
)

plt.plot(
    sample_df["Sample Size"],
    sample_df["Bayesian Mean"],

```

```

        marker="s",
        label="Bayesian Mean"
    )

    plt.axhline(
        mle_mean,
        linestyle="--",
        label="Full Dataset MLE"
    )

    plt.xlabel("Sample Size")
    plt.ylabel("Estimated Mean")
    plt.title("MLE vs Bayesian Estimation")
    plt.legend()
    plt.grid(True)
    plt.show()

plt.figure(figsize=(10, 6))

plt.plot(
    prior_df["Prior Mean"],
    prior_df["Posterior Mean"],
    marker="o"
)

plt.axhline(
    mle_mean,
    linestyle="--",
    label="MLE Mean"
)

plt.xlabel("Prior Mean")
plt.ylabel("Posterior Mean")
plt.title("Effect of Prior Assumptions")
plt.legend()
plt.grid(True)
plt.show()

plt.figure(figsize=(10, 6))

sns.histplot(
    data,
    bins=5,
    kde=True
)

plt.axvline(
    mle_mean,
    linestyle="--",
    label="MLE Mean"

```



```

)

plt.xlabel("Fixed Acidity")
plt.ylabel("Frequency")
plt.title("Fixed Acidity Distribution")
plt.legend()
plt.show()

final_bayesian_mean, final_bayesian_variance = bayesian_estimation(
    data,
    prior_mean=8,
    prior_variance=1
)

final_results = pd.DataFrame({
    "Method": [
        "MLE",
        "Bayesian"
    ],
    "Mean": [
        mle_mean,
        final_bayesian_mean
    ],
    "Variance": [
        mle_variance,
        final_bayesian_variance
    ]
})

print("\nFinal MLE vs Bayesian Comparison")
display(final_results)

```

## Output

\*\*\* Dataset:

|   | fixed acidity | volatile acidity | citric acid | residual sugar | chlorides | free sulfur dioxide | total sulfur dioxide | density |
|---|---------------|------------------|-------------|----------------|-----------|---------------------|----------------------|---------|
| 0 | 7.4           | 0.70             | 0.00        | 1.9            | 0.076     | 11                  | 34                   | 0.9978  |
| 1 | 7.8           | 0.88             | 0.00        | 2.6            | 0.098     | 25                  | 67                   | 0.9968  |
| 2 | 7.8           | 0.76             | 0.04        | 2.3            | 0.092     | 15                  | 54                   | 0.9970  |
| 3 | 11.2          | 0.28             | 0.56        | 1.9            | 0.075     | 17                  | 60                   | 0.9980  |
| 4 | 7.4           | 0.70             | 0.00        | 1.9            | 0.076     | 11                  | 34                   | 0.9978  |
| 5 | 7.4           | 0.66             | 0.00        | 1.8            | 0.075     | 13                  | 40                   | 0.9978  |
| 6 | 7.9           | 0.80             | 0.06        | 1.6            | 0.089     | 15                  | 59                   | 0.9964  |
| 7 | 7.3           | 0.65             | 0.00        | 1.2            | 0.065     | 15                  | 21                   | 0.9946  |
| 8 | 7.8           | 0.58             | 0.02        | 2.0            | 0.073     | 9                   | 18                   | 0.9968  |
| 9 | 6.7           | 0.58             | 0.08        | 1.8            | 0.097     | 15                  | 65                   | 0.9959  |

Shape: (10, 8)

Statistics:

|       | fixed acidity | volatile acidity | citric acid | residual sugar | chlorides | free sulfur dioxide | total sulfur dioxide | density   |
|-------|---------------|------------------|-------------|----------------|-----------|---------------------|----------------------|-----------|
| count | 10.000000     | 10.000000        | 10.000000   | 10.000000      | 10.000000 | 10.000000           | 10.000000            | 10.000000 |
| mean  | 7.870000      | 0.639000         | 0.076000    | 1.900000       | 0.079600  | 14.600000           | 45.200000            | 0.996890  |
| std   | 1.222975      | 0.155867         | 0.172511    | 0.374186       | 0.011702  | 4.402020            | 18.127940            | 0.001065  |
| min   | 6.700000      | 0.280000         | 0.000000    | 1.200000       | 0.065000  | 9.000000            | 18.000000            | 0.994600  |
| 25%   | 7.400000      | 0.585000         | 0.000000    | 1.800000       | 0.073500  | 11.500000           | 34.000000            | 0.996500  |
| 50%   | 7.600000      | 0.655000         | 0.010000    | 1.900000       | 0.075500  | 15.000000           | 47.000000            | 0.996900  |
| 75%   | 7.800000      | 0.700000         | 0.055000    | 1.975000       | 0.088000  | 15.000000           | 59.750000            | 0.997800  |
| max   | 11.200000     | 0.880000         | 0.560000    | 2.600000       | 0.098000  | 25.000000           | 67.000000            | 0.998000  |

MLE Estimation  
MLE Mean: 7.87  
MLE Variance: 1.346899999999994

Sample Estimation  
Sample Mean: 7.87  
Sample Variance: 1.4956666666666658

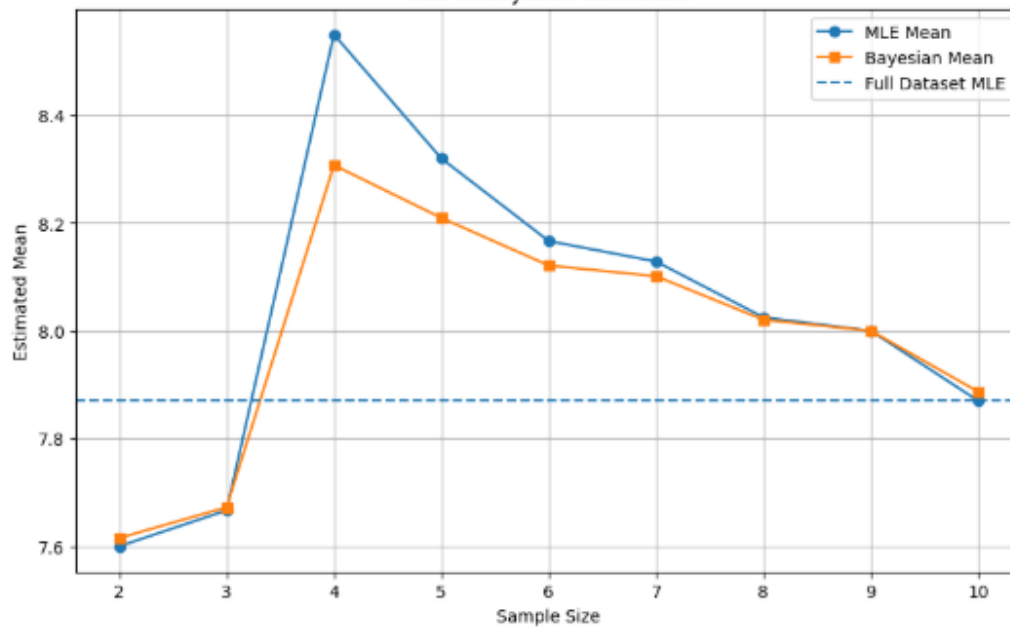
Bayesian Estimation with Different Priors

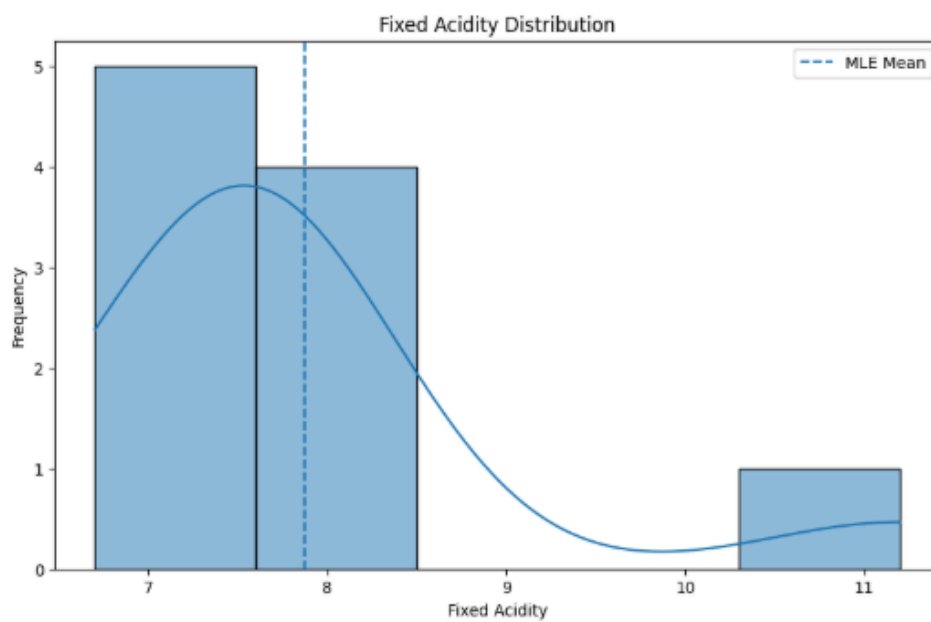
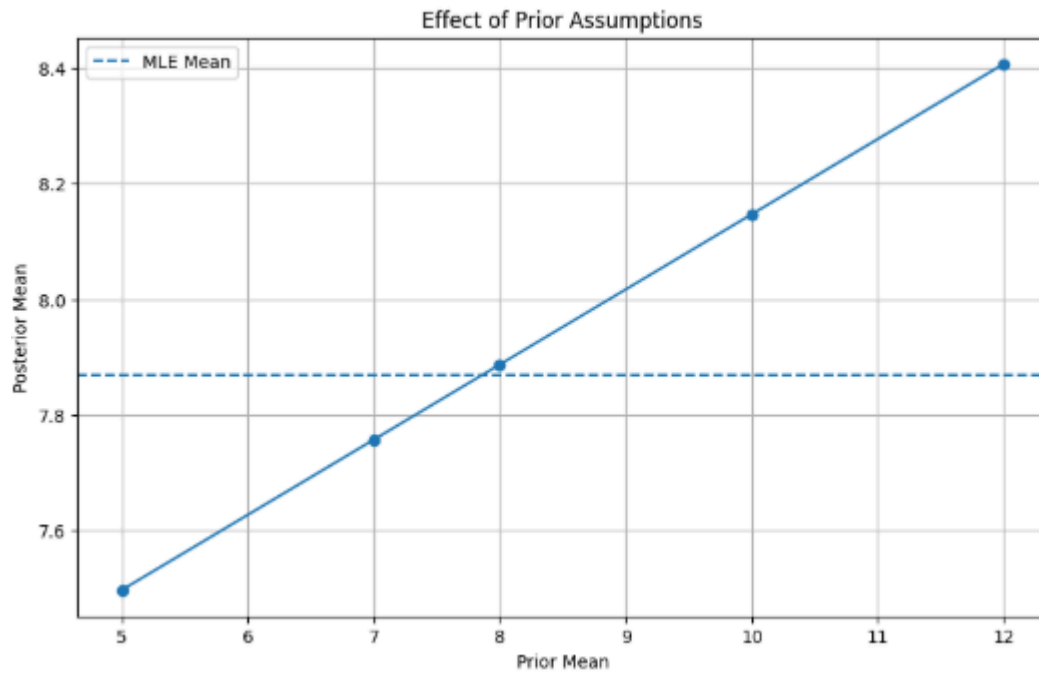
|   | Prior Mean | Prior Variance | Posterior Mean | Posterior Variance |
|---|------------|----------------|----------------|--------------------|
| 0 | 5          | 1              | 7.486593       | 0.130107           |
| 1 | 7          | 1              | 7.756807       | 0.130107           |
| 2 | 8          | 1              | 7.886914       | 0.130107           |
| 3 | 10         | 1              | 8.147128       | 0.130107           |
| 4 | 12         | 1              | 8.407342       | 0.130107           |

Small vs Large Sample Size

|   | Sample Size | MLE Mean | Bayesian Mean | MLE Variance | Bayesian Variance |
|---|-------------|----------|---------------|--------------|-------------------|
| 0 | 2           | 7.600000 | 7.615385      | 0.040000     | 0.038462          |
| 1 | 3           | 7.666667 | 7.672489      | 0.035556     | 0.017467          |
| 2 | 4           | 8.550000 | 8.307406      | 2.367500     | 0.441081          |
| 3 | 5           | 8.320000 | 8.209644      | 2.105600     | 0.344864          |
| 4 | 6           | 8.166667 | 8.121261      | 1.872222     | 0.272433          |
| 5 | 7           | 8.128571 | 8.101324      | 1.613469     | 0.211923          |
| 6 | 8           | 8.025000 | 8.026620      | 1.486875     | 0.175197          |
| 7 | 9           | 8.000000 | 8.000000      | 1.326667     | 0.142244          |
| 8 | 10          | 7.870000 | 7.886914      | 1.346100     | 0.130107          |

MLE vs Bayesian Estimation





Final MLE vs Bayesian Comparison

|   | Method   | Mean     | Variance |
|---|----------|----------|----------|
| 0 | MLE      | 7.870000 | 1.346100 |
| 1 | Bayesian | 7.888914 | 0.130107 |

## Experiment 3 — Expectation-Maximization

### Mall Customer Segmentation Dataset

| CustomerID | Gender | Age | Annual Income (k\$) | Spending Score (1-100) |
|------------|--------|-----|---------------------|------------------------|
| 1000       | M      | 39  | 59.9                | 58                     |
| 1001       | M      | 34  | 48.4                | 37                     |
| 1002       | F      | 40  | 70.5                | 26                     |
| 1003       | F      | 47  | 81.1                | 30                     |
| 1004       | F      | 33  | 42.1                | 58                     |
| 1005       | M      | 33  | 36.7                | 73                     |
| 1006       | F      | 48  | 90.4                | 34                     |
| 1007       | F      | 41  | 74.8                | 35                     |
| 1008       | F      | 31  | 42.3                | 43                     |
| 1009       | M      | 39  | 48.6                | 61                     |

### Code

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from sklearn.preprocessing import StandardScaler
from sklearn.mixture import GaussianMixture

data = {
    "CustomerID": [1000, 1001, 1002, 1003, 1004, 1005, 1006, 1007, 1008, 1009],
    "Gender": ["M", "M", "F", "F", "F", "M", "F", "F", "F", "M"],
    "Age": [39, 34, 40, 47, 33, 33, 48, 41, 31, 39],
    "Annual Income (k$)": [59.9, 48.4, 70.5, 81.1, 42.1, 36.7, 90.4, 74.8, 42.3, 48.6],
    "Spending Score (1-100)": [58, 37, 26, 30, 58, 73, 34, 35, 43, 61]
}

df = pd.DataFrame(data)

print("Dataset:")
display(df)
```

```

print("Shape:", df.shape)

print("\nDataset Information:")
print(df.info())

print("\nStatistical Summary:")
display(df.describe())

X = df[
    [
        "Annual Income (k$)",
        "Spending Score (1-100)"
    ]
]

scaler = StandardScaler()

X_scaled = scaler.fit_transform(X)

print("\nScaled Data:")
display(pd.DataFrame(
    X_scaled,
    columns=[
        "Annual Income",
        "Spending Score"
    ]
))

cluster_results = []

for k in [2, 3, 4, 5, 6]:

    model = GaussianMixture(
        n_components=k,
        covariance_type="full",
        init_params="kmeans",
        n_init=1,
        max_iter=300,
        random_state=42
    )

    model.fit(X_scaled)

    cluster_results.append({
        "Clusters": k,
        "Iterations": model.n_iter_,
        "Log Likelihood": model.lower_bound_,
        "AIC": model.aic(X_scaled),
        "BIC": model.bic(X_scaled)
    })

```

```

cluster_df = pd.DataFrame(cluster_results)

print("\nDifferent Number of Clusters:")
display(cluster_df)

plt.figure(figsize=(10, 6))

plt.plot(
    cluster_df["Clusters"],
    cluster_df["Log Likelihood"],
    marker="o"
)

plt.xlabel("Number of Clusters")
plt.ylabel("Log Likelihood")
plt.title("EM Log Likelihood for Different Cluster Numbers")
plt.grid(True)
plt.show()

plt.figure(figsize=(10, 6))

plt.plot(
    cluster_df["Clusters"],
    cluster_df["AIC"],
    marker="o",
    label="AIC"
)

plt.plot(
    cluster_df["Clusters"],
    cluster_df["BIC"],
    marker="s",
    label="BIC"
)

plt.xlabel("Number of Clusters")
plt.ylabel("Score")
plt.title("AIC and BIC for Different Cluster Numbers")
plt.legend()
plt.grid(True)
plt.show()

model = GaussianMixture(
    n_components=3,
    covariance_type="full",
    init_params="kmeans",
    n_init=10,
    max_iter=300,
    random_state=42
)

```

```

)

labels = model.fit_predict(X_scaled)

df["Cluster"] = labels

print("\nClustered Dataset:")
display(df)

plt.figure(figsize=(10, 6))

plt.scatter(
    X_scaled[:, 0],
    X_scaled[:, 1],
    c=labels,
    s=100
)

plt.xlabel("Annual Income")
plt.ylabel("Spending Score")
plt.title("EM Gaussian Mixture Clustering")
plt.grid(True)
plt.show()

initialization_results = []

for seed in range(10):

    model = GaussianMixture(
        n_components=3,
        covariance_type="full",
        init_params="random",
        n_init=1,
        max_iter=300,
        random_state=seed
    )

    model.fit(X_scaled)

    initialization_results.append({
        "Random Seed": seed,
        "Iterations": model.n_iter_,
        "Final Log Likelihood": model.lower_bound_,
        "AIC": model.aic(X_scaled),
        "BIC": model.bic(X_scaled),
        "Converged": model.converged_
    })

initialization_df = pd.DataFrame(
    initialization_results

```

```

)

print("\nRandom Initialization Results:")
display(initialization_df)

kmeans_results = []

for seed in range(10):

    model = GaussianMixture(
        n_components=3,
        covariance_type="full",
        init_params="kmeans",
        n_init=1,
        max_iter=300,
        random_state=seed
    )

    model.fit(X_scaled)

    kmeans_results.append({
        "Random Seed": seed,
        "Iterations": model.n_iter_,
        "Final Log Likelihood": model.lower_bound_,
        "AIC": model.aic(X_scaled),
        "BIC": model.bic(X_scaled),
        "Converged": model.converged_
    })

kmeans_df = pd.DataFrame(kmeans_results)

print("\nK-Means Initialization Results:")
display(kmeans_df)

comparison_df = pd.DataFrame({
    "Initialization": ["Random", "K-Means"],
    "Average Iterations": [
        initialization_df["Iterations"].mean(),
        kmeans_df["Iterations"].mean()
    ],
    "Average Log Likelihood": [
        initialization_df["Final Log Likelihood"].mean(),
        kmeans_df["Final Log Likelihood"].mean()
    ],
    "Best Log Likelihood": [
        initialization_df["Final Log Likelihood"].max(),
        kmeans_df["Final Log Likelihood"].max()
    ],
    "Average AIC": [
        initialization_df["AIC"].mean(),

```



```

        kmeans_df["AIC"].mean()
    ],
    "Average BIC": [
        initialization_df["BIC"].mean(),
        kmeans_df["BIC"].mean()
    ]
})

print("\nInitialization Comparison:")
display(comparison_df)

plt.figure(figsize=(10, 6))

plt.plot(
    initialization_df["Random Seed"],
    initialization_df["Final Log Likelihood"],
    marker="o",
    label="Random Initialization"
)

plt.plot(
    kmeans_df["Random Seed"],
    kmeans_df["Final Log Likelihood"],
    marker="s",
    label="K-Means Initialization"
)

plt.xlabel("Random Seed")
plt.ylabel("Final Log Likelihood")
plt.title("Effect of Initialization on EM Convergence")
plt.legend()
plt.grid(True)
plt.show()

plt.figure(figsize=(10, 6))

plt.plot(
    initialization_df["Random Seed"],
    initialization_df["Iterations"],
    marker="o",
    label="Random Initialization"
)

plt.plot(
    kmeans_df["Random Seed"],
    kmeans_df["Iterations"],
    marker="s",
    label="K-Means Initialization"
)

```

```

plt.xlabel("Random Seed")
plt.ylabel("Number of Iterations")
plt.title("EM Convergence Speed")
plt.legend()
plt.grid(True)
plt.show()

print("\nLocal Optimum Analysis:")

print(
    "Random initialization likelihood range:",
    initialization_df["Final Log Likelihood"].min(),
    "to",
    initialization_df["Final Log Likelihood"].max()
)

print(
    "K-Means initialization likelihood range:",
    kmeans_df["Final Log Likelihood"].min(),
    "to",
    kmeans_df["Final Log Likelihood"].max()
)

best_model = GaussianMixture(
    n_components=3,
    covariance_type="full",
    init_params="kmeans",
    n_init=20,
    max_iter=500,
    random_state=42
)

best_labels = best_model.fit_predict(X_scaled)

print("\nBest Model")
print("Clusters:", best_model.n_components)
print("Iterations:", best_model.n_iter_)
print("Converged:", best_model.converged_)
print("Log Likelihood:", best_model.lower_bound_)
print("AIC:", best_model.aic(X_scaled))
print("BIC:", best_model.bic(X_scaled))

df["Best Cluster"] = best_labels

print("\nFinal Clustered Dataset:")
display(df)

```

# Output

\*\*\* Dataset:

|   | CustomerID | Gender | Age | Annual Income (k\$) | Spending Score (1-100) |
|---|------------|--------|-----|---------------------|------------------------|
| 0 | 1000       | M      | 39  | 59.9                | 58                     |
| 1 | 1001       | M      | 34  | 48.4                | 37                     |
| 2 | 1002       | F      | 40  | 70.5                | 26                     |
| 3 | 1003       | F      | 47  | 81.1                | 30                     |
| 4 | 1004       | F      | 33  | 42.1                | 58                     |
| 5 | 1005       | M      | 33  | 38.7                | 73                     |
| 6 | 1006       | F      | 48  | 90.4                | 34                     |
| 7 | 1007       | F      | 41  | 74.8                | 35                     |
| 8 | 1008       | F      | 31  | 42.3                | 43                     |
| 9 | 1009       | M      | 39  | 48.6                | 61                     |

Shape: (10, 5)

Dataset Information:

<class 'pandas.core.frame.DataFrame'>

RangeIndex: 10 entries, 0 to 9

Data columns (total 5 columns):

| # | Column                 | Non-Null Count | Dtype   |
|---|------------------------|----------------|---------|
| 0 | CustomerID             | 10 non-null    | int64   |
| 1 | Gender                 | 10 non-null    | object  |
| 2 | Age                    | 10 non-null    | int64   |
| 3 | Annual Income (k\$)    | 10 non-null    | float64 |
| 4 | Spending Score (1-100) | 10 non-null    | int64   |

dtypes: float64(1), int64(3), object(1)

memory usage: 532.0+ bytes

None

Statistical Summary:

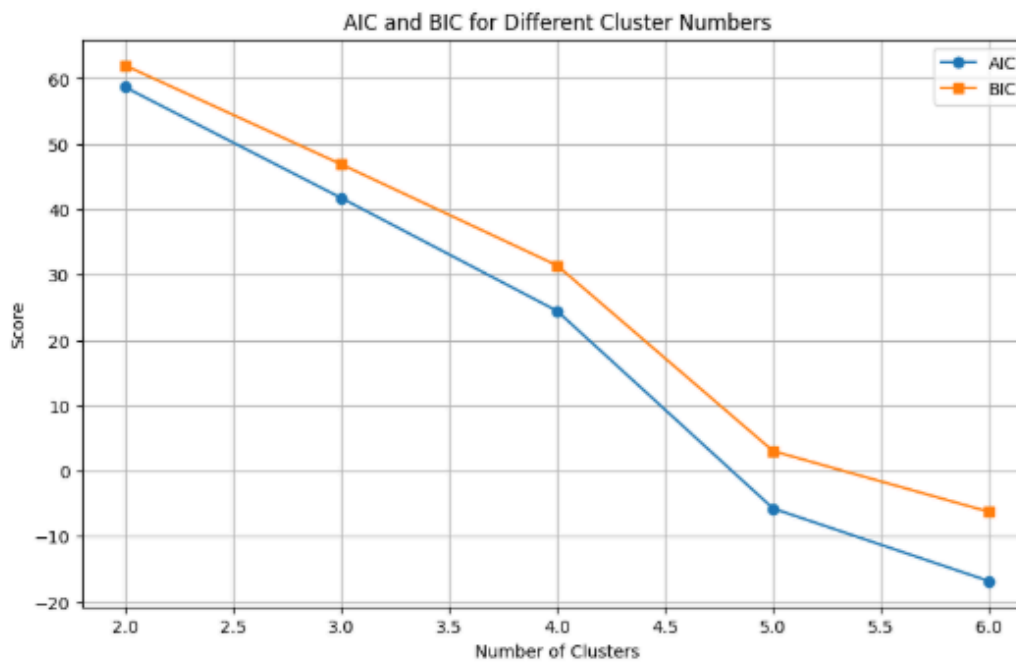
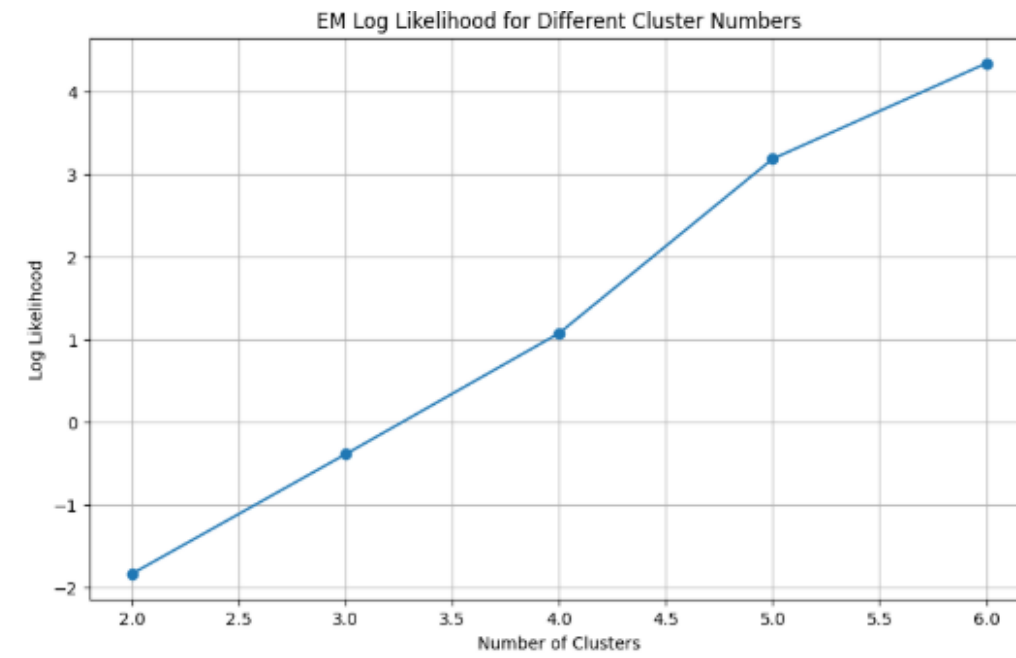
|       | CustomerID  | Age       | Annual Income (k\$) | Spending Score (1-100) |
|-------|-------------|-----------|---------------------|------------------------|
| count | 10.000000   | 10.000000 | 10.000000           | 10.000000              |
| mean  | 1004.500000 | 38.500000 | 59.480000           | 45.500000              |
| std   | 3.02765     | 5.854723  | 18.869889           | 15.813145              |
| min   | 1000.000000 | 31.000000 | 38.700000           | 26.000000              |
| 25%   | 1002.250000 | 33.250000 | 43.825000           | 34.250000              |
| 50%   | 1004.500000 | 39.000000 | 54.250000           | 40.000000              |
| 75%   | 1006.750000 | 40.750000 | 73.725000           | 58.000000              |
| max   | 1009.000000 | 48.000000 | 90.400000           | 73.000000              |

Scaled Data:

|   | Annual Income | Spending Score |
|---|---------------|----------------|
| 0 | 0.023713      | 0.833241       |
| 1 | -0.625572     | -0.566804      |
| 2 | 0.622184      | -1.299856      |
| 3 | 1.220656      | -1.033219      |
| 4 | -0.981267     | 0.833241       |
| 5 | -1.288149     | 1.833130       |
| 6 | 1.745730      | -0.786581      |
| 7 | 0.864960      | -0.699922      |
| 8 | -0.969975     | -0.166648      |
| 9 | -0.614280     | 1.033219       |

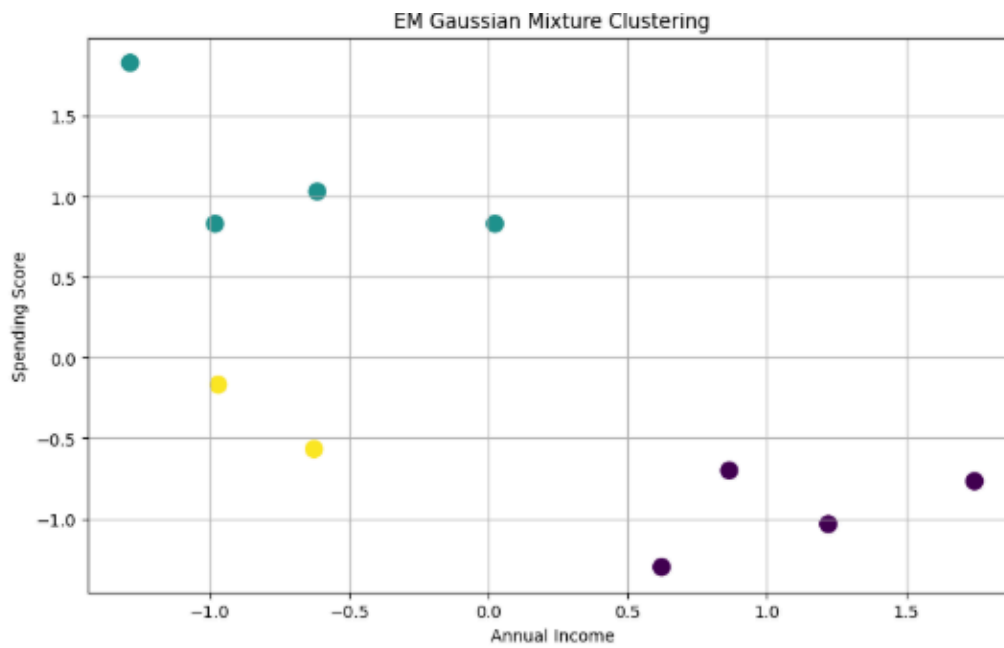
Different Number of Clusters:

|   | Clusters | Iterations | Log Likelihood | AIC        | BIC       |
|---|----------|------------|----------------|------------|-----------|
| 0 | 2        | 2          | -1.834082      | 58.681637  | 62.010073 |
| 1 | 3        | 2          | -0.388000      | 41.760008  | 46.903954 |
| 2 | 4        | 2          | 1.076213       | 24.475742  | 31.435199 |
| 3 | 5        | 2          | 3.185726       | -5.714512  | 3.060456  |
| 4 | 6        | 2          | 4.341031       | -16.820629 | -6.230150 |



Clustered Dataset:

|   | CustomerID | Gender | Age | Annual Income (k\$) | Spending Score (1-100) | Cluster |
|---|------------|--------|-----|---------------------|------------------------|---------|
| 0 | 1000       | M      | 39  | 59.9                | 58                     | 1       |
| 1 | 1001       | M      | 34  | 48.4                | 37                     | 2       |
| 2 | 1002       | F      | 40  | 70.6                | 28                     | 0       |
| 3 | 1003       | F      | 47  | 81.1                | 30                     | 0       |
| 4 | 1004       | F      | 33  | 42.1                | 58                     | 1       |
| 5 | 1005       | M      | 33  | 38.7                | 73                     | 1       |
| 6 | 1006       | F      | 48  | 90.4                | 34                     | 0       |
| 7 | 1007       | F      | 41  | 74.8                | 35                     | 0       |
| 8 | 1008       | F      | 31  | 42.3                | 43                     | 2       |
| 9 | 1009       | M      | 39  | 48.6                | 61                     | 1       |

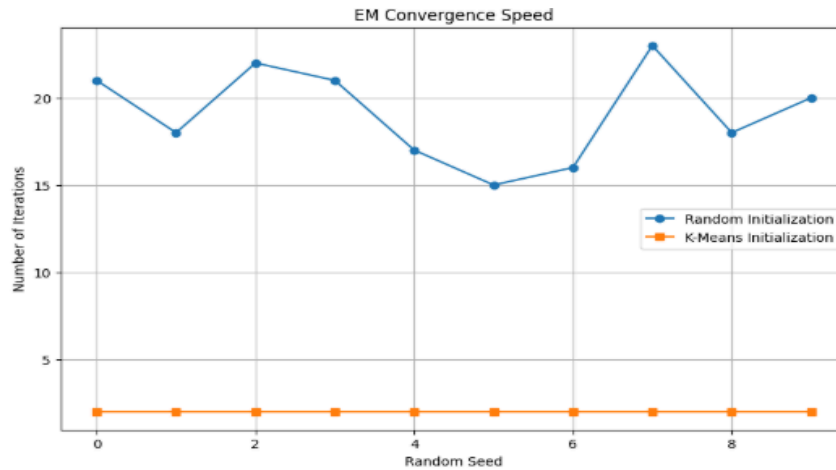


Random Initialization Results:

| Random Seed | Iterations | Final Log Likelihood | AIC       | BIC       | Converged |      |
|-------------|------------|----------------------|-----------|-----------|-----------|------|
| 0           | 0          | 21                   | -0.769454 | 49.389079 | 54.533025 | True |
| 1           | 1          | 18                   | -1.331413 | 60.627967 | 65.771913 | True |
| 2           | 2          | 22                   | -1.242190 | 58.843713 | 63.987660 | True |
| 3           | 3          | 21                   | -0.645202 | 46.904046 | 52.047992 | True |
| 4           | 4          | 17                   | -0.241991 | 38.839814 | 43.983761 | True |
| 5           | 5          | 15                   | -1.356649 | 61.132973 | 66.276920 | True |
| 6           | 6          | 16                   | -1.356649 | 61.132973 | 66.276920 | True |
| 7           | 7          | 23                   | -0.241991 | 38.839814 | 43.983761 | True |
| 8           | 8          | 18                   | -1.356649 | 61.132973 | 66.276920 | True |
| 9           | 9          | 20                   | 0.324892  | 27.502154 | 32.646101 | True |

K-Means Initialization Results:

| Random Seed | Iterations | Final Log Likelihood | AIC       | BIC       | Converged |      |
|-------------|------------|----------------------|-----------|-----------|-----------|------|
| 0           | 0          | 2                    | -0.561216 | 45.224315 | 50.368262 | True |
| 1           | 1          | 2                    | -0.388000 | 41.760008 | 46.903954 | True |
| 2           | 2          | 2                    | -0.388000 | 41.760008 | 46.903954 | True |
| 3           | 3          | 2                    | -1.603939 | 66.078687 | 71.222633 | True |
| 4           | 4          | 2                    | -0.388000 | 41.760008 | 46.903954 | True |
| 5           | 5          | 2                    | 0.333094  | 27.338127 | 32.482074 | True |
| 6           | 6          | 2                    | -1.603939 | 66.078687 | 71.222633 | True |
| 7           | 7          | 2                    | -1.603939 | 66.078687 | 71.222633 | True |
| 8           | 8          | 2                    | -1.603939 | 66.078687 | 71.222633 | True |
| 9           | 9          | 2                    | -0.388000 | 41.760008 | 46.903954 | True |

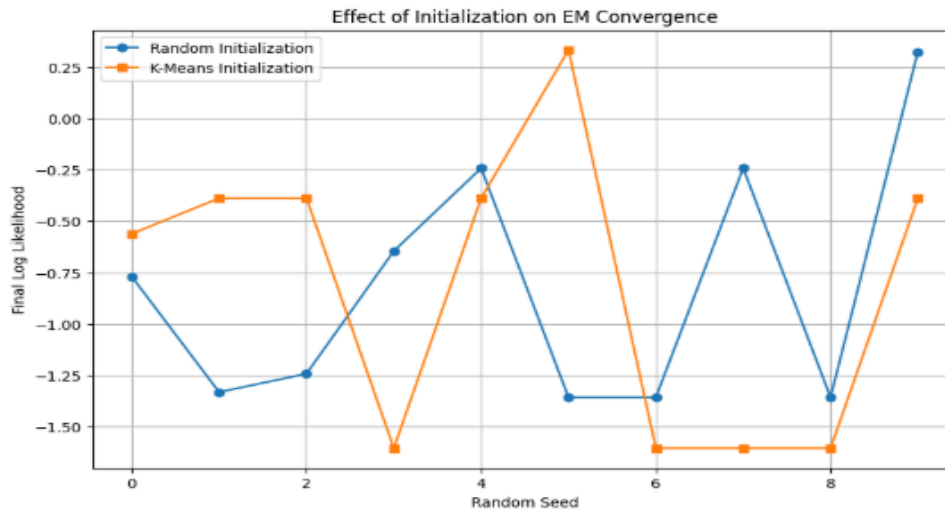


Local Optimum Analysis:  
Random initialization likelihood range: -1.3566487291641154 to 0.3248922927473032  
K-Means initialization likelihood range: -1.6039394795803599 to 0.33309364441085176

Best Model  
Clusters: 3  
Iterations: 2  
Converged: True  
Log Likelihood: -0.38800037911881535  
AIC: 41.7600075823707  
BIC: 46.903954163269475

Initialization Comparison:

|   | Initialization | Average Iterations | Average Log Likelihood | Best Log Likelihood | Average AIC | Average BIC |
|---|----------------|--------------------|------------------------|---------------------|-------------|-------------|
| 0 | Random         | 19.1               | -0.821729              | 0.324892            | 50.434551   | 55.578497   |
| 1 | K-Means        | 2.0                | -0.819588              | 0.333094            | 50.301722   | 55.535088   |



Local Optimum Analysis:  
Random initialization likelihood range: -1.3566487291641154 to 0.3248922927473032  
K-Means initialization likelihood range: -1.6039394795803599 to 0.33309364441085176

Best Model  
Clusters: 3  
Iterations: 2  
Converged: True  
Log Likelihood: -0.38800037911881535  
AIC: 41.7600075823707  
BIC: 46.903954163269475

Final Clustered Dataset:

|   | CustomerID | Gender | Age | Annual Income (k\$) | Spending Score (1-100) | Cluster | Best Cluster |
|---|------------|--------|-----|---------------------|------------------------|---------|--------------|
| 0 | 1000       | M      | 39  | 59.9                | 58                     | 1       | 1            |
| 1 | 1001       | M      | 34  | 48.4                | 37                     | 2       | 2            |
| 2 | 1002       | F      | 40  | 70.5                | 26                     | 0       | 0            |
| 3 | 1003       | F      | 47  | 81.1                | 30                     | 0       | 0            |
| 4 | 1004       | F      | 33  | 42.1                | 58                     | 1       | 1            |
| 5 | 1005       | M      | 33  | 38.7                | 73                     | 1       | 1            |
| 6 | 1006       | F      | 48  | 90.4                | 34                     | 0       | 0            |
| 7 | 1007       | F      | 41  | 74.8                | 35                     | 0       | 0            |
| 8 | 1008       | F      | 31  | 42.3                | 43                     | 2       | 2            |
| 9 | 1009       | M      | 39  | 48.6                | 61                     | 1       | 1            |

## Experiment 4 — Bayesian Belief Network

### Heart Disease Dataset

| Diastolic_BP | BMI | Smoking | Heart_Rate | Blood_Sugar_Fasting | Cholesterol_Total | Heart_Disease | Diastolic_BP |
|--------------|-----|---------|------------|---------------------|-------------------|---------------|--------------|
| 104          | 157 | 26.4    | 99         | 71                  | 165               | 200           | 104          |
| 111          | 163 | 33      | 72         | 60                  | 145               | 206           | 111          |
| 116          | 152 | 32.3    | 102        | 78                  | 148               | 208           | 116          |
| 171          | 171 | 37.4    | 92         | 109                 | 105               | 290           | 171          |

### Code

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

!pip -q install pgmpy

from pgmpy.models import DiscreteBayesianNetwork
from pgmpy.estimators import MaximumLikelihoodEstimator
from pgmpy.inference import VariableElimination

data = {
    "Diastolic_BP": [104, 111, 116, 171],
    "BMI": [157, 163, 152, 171],
    "Smoking": [26.4, 33, 32.3, 37.4],
    "Heart_Rate": [99, 72, 102, 92],
    "Blood_Sugar_Fasting": [71, 60, 78, 109],
    "Cholesterol_Total": [165, 145, 148, 105],
    "Heart_Disease": [200, 206, 208, 290]
}

df = pd.DataFrame(data)

print("Dataset:")
display(df)

print("Shape:", df.shape)

print("\nStatistical Summary:")
display(df.describe())
```

```
df["Diastolic_BP_Level"] = pd.cut(
    df["Diastolic_BP"],
    bins=[0, 100, 120, 200],
    labels=["Normal", "High", "Very_High"]
)
```

```
df["BMI_Level"] = pd.cut(
    df["BMI"],
    bins=[0, 160, 170, 250],
    labels=["Low", "Medium", "High"]
)
```

```
df["Smoking_Level"] = pd.cut(
    df["Smoking"],
    bins=[0, 25, 35, 100],
    labels=["Low", "Medium", "High"]
)
```

```
df["Heart_Rate_Level"] = pd.cut(
    df["Heart_Rate"],
    bins=[0, 80, 100, 200],
    labels=["Low", "Normal", "High"]
)
```

```
df["Blood_Sugar_Level"] = pd.cut(
    df["Blood_Sugar_Fasting"],
    bins=[0, 70, 100, 200],
    labels=["Low", "Normal", "High"]
)
```

```
df["Cholesterol_Level"] = pd.cut(
    df["Cholesterol_Total"],
    bins=[0, 150, 200, 400],
    labels=["Low", "Medium", "High"]
)
```

```
df["Heart_Disease_Level"] = pd.cut(
    df["Heart_Disease"],
    bins=[0, 210, 250, 400],
    labels=["Low", "Medium", "High"]
)
```

```
bn_data = df[
    "Diastolic_BP_Level",
    "BMI_Level",
    "Smoking_Level",
    "Heart_Rate_Level",
    "Blood_Sugar_Level",
    "Cholesterol_Level",
]
```



```

        "Heart_Disease_Level"
    ]
].dropna()

for column in bn_data.columns:
    bn_data[column] = bn_data[column].astype(str)

print("\nDiscretized Dataset:")
display(bn_data)

model = DiscreteBayesianNetwork([
    ("Diastolic_BP_Level", "Heart_Disease_Level"),
    ("BMI_Level", "Heart_Disease_Level"),
    ("Smoking_Level", "Heart_Disease_Level"),
    ("Heart_Rate_Level", "Heart_Disease_Level"),
    ("Blood_Sugar_Level", "Heart_Disease_Level"),
    ("Cholesterol_Level", "Heart_Disease_Level")
])

model.fit(
    bn_data,
    estimator=MaximumLikelihoodEstimator
)

print("\nBayesian Network:")
print(model.edges())

print("\nModel Validity:")
print(model.check_model())

inference = VariableElimination(model)

result = inference.query(
    variables=["Heart_Disease_Level"]
)

print("\nPosterior Probability Without Evidence:")
print(result)

evidence_cases = [
    {
        "Diastolic_BP_Level": "High"
    },
    {
        "BMI_Level": "Medium"
    },
    {
        "Heart_Rate_Level": "Normal"
    },
    {

```

```

        "Blood_Sugar_Level": "High"
    },
    {
        "Cholesterol_Level": "Low"
    }
]

results = []

for evidence in evidence_cases:

    try:
        result = inference.query(
            variables=["Heart_Disease_Level"],
            evidence=evidence
        )

        values = result.values

        results.append({
            "Evidence": str(evidence),
            "Posterior Probability": values.max()
        })

        print("\nEvidence:", evidence)
        print(result)

    except Exception as e:
        print("\nEvidence:", evidence)
        print("Inference Error:", e)

evidence_df = pd.DataFrame(results)

print("\nEvidence Comparison:")
display(evidence_df)

multi_evidence = {
    "Diastolic_BP_Level": "High",
    "BMI_Level": "Medium",
    "Heart_Rate_Level": "Normal",
    "Blood_Sugar_Level": "High",
    "Cholesterol_Level": "Low"
}

result = inference.query(
    variables=["Heart_Disease_Level"],
    evidence=multi_evidence
)

print("\nMultiple Evidence Inference:")

```

```

print(result)
plt.figure(figsize=(10, 6))
plt.bar(
    range(len(evidence_df)),
    evidence_df["Posterior Probability"]
)
plt.xticks(
    range(len(evidence_df)),
    evidence_df["Evidence"],
    rotation=30,
    ha="right"
)
plt.ylabel("Posterior Probability")
plt.xlabel("Evidence Condition")
plt.title("Effect of Evidence on Posterior Probability")
plt.tight_layout()
plt.show()
plt.figure(figsize=(10, 6))
plt.bar(
    df["Heart_Disease_Level"].astype(str),
    df["Blood_Sugar_Level"].astype(str)
)
plt.xlabel("Heart Disease Level")
plt.ylabel("Blood Sugar Level")
plt.title("Relationship Between Blood Sugar and Heart Disease")
plt.show()

```

## Output

Dataset:

|   | Diastolic_BP | BMI | Smoking | Heart_Rate | Blood_Sugar_Fasting | Cholesterol_Total | Heart_Disease |
|---|--------------|-----|---------|------------|---------------------|-------------------|---------------|
| 0 | 104          | 157 | 28.4    | 99         | 71                  | 185               | 200           |
| 1 | 111          | 183 | 33.0    | 72         | 80                  | 145               | 208           |
| 2 | 118          | 152 | 32.3    | 102        | 78                  | 148               | 208           |
| 3 | 171          | 171 | 37.4    | 92         | 108                 | 105               | 290           |

Shape: (4, 7)

Statistical Summary:

|       | Diastolic_BP | BMI        | Smoking   | Heart_Rate | Blood_Sugar_Fasting | Cholesterol_Total | Heart_Disease |
|-------|--------------|------------|-----------|------------|---------------------|-------------------|---------------|
| count | 4.000000     | 4.000000   | 4.000000  | 4.00       | 4.000000            | 4.000000          | 4.000000      |
| mean  | 125.500000   | 180.750000 | 32.275000 | 91.25      | 79.500000           | 140.750000        | 228.000000    |
| std   | 30.730007    | 8.180261   | 4.520801  | 13.50      | 21.015887           | 25.408332         | 42.801889     |
| min   | 104.000000   | 152.000000 | 28.400000 | 72.00      | 80.000000           | 105.000000        | 200.000000    |
| 25%   | 109.250000   | 155.750000 | 30.825000 | 87.00      | 88.250000           | 135.000000        | 204.500000    |
| 50%   | 113.500000   | 180.000000 | 32.850000 | 95.50      | 74.500000           | 149.500000        | 207.000000    |
| 75%   | 129.750000   | 185.000000 | 34.100000 | 98.75      | 85.750000           | 152.250000        | 228.500000    |
| max   | 171.000000   | 171.000000 | 37.400000 | 102.00     | 108.000000          | 185.000000        | 290.000000    |

Discretized Dataset:

|   | Diastolic_BP_Level | BMI_Level | Smoking_Level | Heart_Rate_Level | Blood_Sugar_Level | Cholesterol_Level | Heart_Disease_Level |
|---|--------------------|-----------|---------------|------------------|-------------------|-------------------|---------------------|
| 0 | High               | Low       | Medium        | Normal           | Normal            | Medium            | Low                 |
| 1 | High               | Medium    | Medium        | Low              | Low               | Low               | Low                 |
| 2 | High               | Low       | Medium        | High             | Normal            | Low               | Low                 |
| 3 | Very_High          | High      | High          | Normal           | High              | Low               | High                |

/tmp/ipykernel\_2857/345866298.py:180: FutureWarning: 'pgmpy.estimators.MaximumLikelihoodEstimator' is deprecated and will be removed in v1.3.0. Please use 'pgmpy.parameter\_estimator.DiscreteMLE' instead.

mle\_estimator = MaximumLikelihoodEstimator(model=model, data=bn\_data) # Reverted to MaximumLikelihoodEstimator

## Experiment 5 — Decision Tree Classifier

### Pima Indians Diabetes Database

|       | Pregna<br>ncies | Glucos<br>e    | BloodPr<br>essure | SkinThi<br>ckness | Insulin        | BMI            | Diabete<br>sPedigr<br>eeFunct<br>ion | Age            | Outcom<br>e    |
|-------|-----------------|----------------|-------------------|-------------------|----------------|----------------|--------------------------------------|----------------|----------------|
| count | 768.000<br>000  | 768.000<br>000 | 768.000<br>000    | 768.000<br>000    | 768.000<br>000 | 768.000<br>000 | 768.000<br>000                       | 768.000<br>000 | 768.000<br>000 |
| mean  | 3.84505<br>2    | 120.894<br>531 | 69.1054<br>69     | 20.5364<br>58     | 79.7994<br>79  | 31.9925<br>78  | 0.47187<br>6                         | 33.2408<br>85  | 0.34895<br>8   |
| std   | 3.36957<br>8    | 31.9726<br>18  | 19.3558<br>07     | 15.9522<br>18     | 115.244<br>002 | 7.88416<br>0   | 0.33132<br>9                         | 11.7602<br>32  | 0.47695<br>1   |
| min   | 0.00000<br>0    | 0.00000<br>0   | 0.00000<br>0      | 0.00000<br>0      | 0.00000<br>0   | 0.00000<br>0   | 0.07800<br>0                         | 21.0000<br>00  | 0.00000<br>0   |
| 25%   | 1.00000<br>0    | 99.0000<br>00  | 62.0000<br>00     | 0.00000<br>0      | 0.00000<br>0   | 27.3000<br>00  | 0.24375<br>0                         | 24.0000<br>00  | 0.00000<br>0   |
| 50%   | 3.00000<br>0    | 117.000<br>000 | 72.0000<br>00     | 23.0000<br>00     | 30.5000<br>00  | 32.0000<br>00  | 0.37250<br>0                         | 29.0000<br>00  | 0.00000<br>0   |
| 75%   | 6.00000<br>0    | 140.250<br>000 | 80.0000<br>00     | 32.0000<br>00     | 127.250<br>000 | 36.6000<br>00  | 0.62625<br>0                         | 41.0000<br>00  | 1.00000<br>0   |
| max   | 17.0000<br>00   | 199.000<br>000 | 122.000<br>000    | 99.0000<br>00     | 846.000<br>000 | 67.1000<br>00  | 2.42000<br>0                         | 81.0000<br>00  | 1.00000<br>0   |

## Code

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier, plot_tree
from sklearn.metrics import accuracy_score, f1_score, classification_report, confusion_matrix

df = pd.read_csv("diabetes.csv")

print(df.head())
print("\nShape:", df.shape)
print("\nColumns:", df.columns.tolist())
print("\nStatistics:")
display(df.describe())

X = df.drop("Outcome", axis=1)
y = df["Outcome"]

X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.20,
    random_state=42,
    stratify=y
)

gini_model = DecisionTreeClassifier(
    criterion="gini",
    random_state=42
)

gini_model.fit(X_train, y_train)

gini_pred = gini_model.predict(X_test)

gini_accuracy = accuracy_score(
    y_test,
    gini_pred
)

gini_f1 = f1_score(
    y_test,
    gini_pred
)

print("\nGini Index")
```

```

print("Accuracy:", gini_accuracy)
print("F1 Score:", gini_f1)

entropy_model = DecisionTreeClassifier(
    criterion="entropy",
    random_state=42
)

entropy_model.fit(X_train, y_train)

entropy_pred = entropy_model.predict(X_test)

entropy_accuracy = accuracy_score(
    y_test,
    entropy_pred
)

entropy_f1 = f1_score(
    y_test,
    entropy_pred
)

print("\nEntropy")
print("Accuracy:", entropy_accuracy)
print("F1 Score:", entropy_f1)

criterion_results = pd.DataFrame({
    "Criterion": [
        "Gini",
        "Entropy"
    ],
    "Accuracy": [
        gini_accuracy,
        entropy_accuracy
    ],
    "F1 Score": [
        gini_f1,
        entropy_f1
    ]
})

print("\nGini vs Entropy")
display(criterion_results)

criterion_melted = criterion_results.melt(
    id_vars="Criterion",
    var_name="Metric",
    value_name="Score"
)

```

```

plt.figure(figsize=(8, 5))

sns.barplot(
    data=criterion_melted,
    x="Criterion",
    y="Score",
    hue="Metric"
)

plt.ylim(0, 1)
plt.title("Gini vs Entropy")
plt.show()

depth_results = []

for depth in [1, 2, 3, 4, 5, 6, 8, 10, None]:

    model = DecisionTreeClassifier(
        criterion="gini",
        max_depth=depth,
        random_state=42
    )

    model.fit(X_train, y_train)

    train_pred = model.predict(X_train)
    test_pred = model.predict(X_test)

    depth_results.append({
        "Max Depth": str(depth),
        "Training Accuracy": accuracy_score(
            y_train,
            train_pred
        ),
        "Testing Accuracy": accuracy_score(
            y_test,
            test_pred
        ),
        "F1 Score": f1_score(
            y_test,
            test_pred
        )
    })

depth_df = pd.DataFrame(depth_results)

print("\nTree Depth Results")
display(depth_df)

plt.figure(figsize=(10, 6))

```

```

plt.plot(
    depth_df["Max Depth"],
    depth_df["Training Accuracy"],
    marker="o",
    label="Training Accuracy"
)

plt.plot(
    depth_df["Max Depth"],
    depth_df["Testing Accuracy"],
    marker="s",
    label="Testing Accuracy"
)

plt.xlabel("Maximum Tree Depth")
plt.ylabel("Accuracy")
plt.title("Effect of Tree Depth")
plt.legend()
plt.grid(True)
plt.show()

pruning_results = []

for alpha in [
    0,
    0.001,
    0.002,
    0.005,
    0.01,
    0.02,
    0.05
]:

    model = DecisionTreeClassifier(
        criterion="gini",
        ccp_alpha=alpha,
        random_state=42
    )

    model.fit(X_train, y_train)

    train_pred = model.predict(X_train)
    test_pred = model.predict(X_test)

    pruning_results.append({
        "CCP Alpha": alpha,
        "Training Accuracy": accuracy_score(
            y_train,
            train_pred

```



```

    ),
    "Testing Accuracy": accuracy_score(
        y_test,
        test_pred
    ),
    "F1 Score": f1_score(
        y_test,
        test_pred
    ),
    "Tree Depth": model.get_depth(),
    "Number of Leaves": model.get_n_leaves()
})

```

```

pruning_df = pd.DataFrame(pruning_results)

```

```

print("\nPruning Results")
display(pruning_df)

```

```

plt.figure(figsize=(10, 6))

```

```

plt.plot(
    pruning_df["CCP Alpha"],
    pruning_df["Training Accuracy"],
    marker="o",
    label="Training Accuracy"
)

```

```

plt.plot(
    pruning_df["CCP Alpha"],
    pruning_df["Testing Accuracy"],
    marker="s",
    label="Testing Accuracy"
)

```

```

plt.xlabel("CCP Alpha")
plt.ylabel("Accuracy")
plt.title("Effect of Pruning")
plt.legend()
plt.grid(True)
plt.show()

```

```

best_model = DecisionTreeClassifier(
    criterion="gini",
    max_depth=5,
    random_state=42
)

```

```

best_model.fit(X_train, y_train)

```

```

best_pred = best_model.predict(X_test)

```

```
print("\nFinal Decision Tree")
print("Accuracy:", accuracy_score(y_test, best_pred))
print("F1 Score:", f1_score(y_test, best_pred))
```

```
print("\nClassification Report:")
print(
    classification_report(
        y_test,
        best_pred
    )
)
```

```
cm = confusion_matrix(
    y_test,
    best_pred
)
```

```
plt.figure(figsize=(6, 5))
```

```
sns.heatmap(
    cm,
    annot=True,
    fmt="d",
    cmap="Blues",
    xticklabels=[
        "No Diabetes",
        "Diabetes"
    ],
    yticklabels=[
        "No Diabetes",
        "Diabetes"
    ]
)
```

```
plt.xlabel("Predicted")
plt.ylabel("Actual")
plt.title("Decision Tree Confusion Matrix")
plt.show()
```

```
plt.figure(figsize=(22, 10))
```

```
plot_tree(
    best_model,
    feature_names=X.columns,
    class_names=[
        "No Diabetes",
        "Diabetes"
    ],
    filled=True,
```

```

        rounded=True
    )

plt.title("Decision Tree")
plt.show()

feature_importance = pd.DataFrame({
    "Feature": X.columns,
    "Importance": best_model.feature_importances_
})

feature_importance = feature_importance.sort_values(
    "Importance",
    ascending=False
)

print("\nFeature Importance")
display(feature_importance)

plt.figure(figsize=(10, 6))

sns.barplot(
    data=feature_importance,
    x="Importance",
    y="Feature"
)

plt.title("Decision Tree Feature Importance")
plt.show()

final_comparison = pd.DataFrame({
    "Model": [
        "Gini",
        "Entropy",
        "Gini Depth 3",
        "Gini Depth 5",
        "Gini Pruned"
    ],
    "Accuracy": [
        gini_accuracy,
        entropy_accuracy,
        accuracy_score(
            y_test,
            DecisionTreeClassifier(
                criterion="gini",
                max_depth=3,
                random_state=42
            ).fit(X_train, y_train).predict(X_test)
        ),
        accuracy_score(

```

```

        y_test,
        best_pred
    ),
    pruning_df.loc[
        pruning_df["Testing Accuracy"].idxmax(),
        "Testing Accuracy"
    ]
],
"F1 Score": [
    gini_fl,
    entropy_fl,
    f1_score(
        y_test,
        DecisionTreeClassifier(
            criterion="gini",
            max_depth=3,
            random_state=42
        ).fit(X_train, y_train).predict(X_test)
    ),
    f1_score(
        y_test,
        best_pred
    ),
    pruning_df.loc[
        pruning_df["Testing Accuracy"].idxmax(),
        "F1 Score"
    ]
]
})

print("\nFinal Comparison")
display(final_comparison)

```

## Output

Dataset:

|    | Diastolic_BP | BMI | Smoking | Heart_Rate | Blood_Sugar_Fasting | Cholesterol_Total | Heart_Disease |
|----|--------------|-----|---------|------------|---------------------|-------------------|---------------|
| 0  | 104          | 157 | 26.4    | 99         | 71                  | 165               | 200           |
| 1  | 111          | 163 | 33.0    | 72         | 60                  | 145               | 208           |
| 2  | 116          | 152 | 32.3    | 102        | 78                  | 148               | 208           |
| 3  | 171          | 171 | 37.4    | 92         | 109                 | 105               | 290           |
| 4  | 95           | 140 | 20.1    | 65         | 55                  | 180               | 180           |
| 5  | 100          | 165 | 28.5    | 85         | 70                  | 155               | 215           |
| 6  | 120          | 175 | 35.0    | 110        | 115                 | 120               | 300           |
| 7  | 80           | 135 | 18.2    | 60         | 50                  | 190               | 170           |
| 8  | 130          | 180 | 40.0    | 120        | 120                 | 110               | 310           |
| 9  | 105          | 145 | 22.5    | 70         | 62                  | 170               | 190           |
| 10 | 115          | 150 | 30.0    | 95         | 85                  | 160               | 220           |
| 11 | 90           | 160 | 25.0    | 75         | 68                  | 150               | 200           |
| 12 | 140          | 170 | 38.0    | 105        | 110                 | 130               | 320           |
| 13 | 110          | 155 | 29.0    | 80         | 75                  | 175               | 210           |
| 14 | 98           | 168 | 31.0    | 88         | 90                  | 140               | 230           |

Shape: (15, 7)

Statistical Summary:

|       | Diastolic_BP | BMI        | Smoking   | Heart_Rate | Blood_Sugar_Fasting | Cholesterol_Total | Heart_Disease |
|-------|--------------|------------|-----------|------------|---------------------|-------------------|---------------|
| count | 15.000000    | 15.000000  | 15.000000 | 15.000000  | 15.000000           | 15.000000         | 15.000000     |
| mean  | 112.333333   | 159.066667 | 29.760000 | 87.866667  | 81.200000           | 149.533333        | 229.933333    |
| std   | 22.250736    | 13.035483  | 6.507666  | 17.553456  | 22.791602           | 25.248800         | 49.488602     |
| min   | 80.000000    | 135.000000 | 18.200000 | 60.000000  | 50.000000           | 105.000000        | 170.000000    |
| 25%   | 99.000000    | 151.000000 | 25.700000 | 73.500000  | 65.000000           | 135.000000        | 200.000000    |
| 50%   | 110.000000   | 160.000000 | 30.000000 | 88.000000  | 75.000000           | 150.000000        | 210.000000    |
| 75%   | 118.000000   | 169.000000 | 34.000000 | 100.500000 | 99.500000           | 167.500000        | 280.000000    |
| max   | 171.000000   | 180.000000 | 40.000000 | 120.000000 | 120.000000          | 190.000000        | 320.000000    |

Discretized Dataset:

|    | Diastolic_BP_Level | BMI_Level | Smoking_Level | Heart_Rate_Level | Blood_Sugar_Level | Cholesterol_Level | Heart_Disease_Level |
|----|--------------------|-----------|---------------|------------------|-------------------|-------------------|---------------------|
| 0  | High               | Low       | Medium        | Normal           | Normal            | Medium            | Low                 |
| 1  | High               | Medium    | Medium        | Low              | Low               | Low               | Low                 |
| 2  | High               | Low       | Medium        | High             | Normal            | Low               | Low                 |
| 3  | Very_High          | High      | High          | Normal           | High              | Low               | High                |
| 4  | Normal             | Low       | Low           | Low              | Low               | Medium            | Low                 |
| 5  | Normal             | Medium    | Medium        | Normal           | Low               | Medium            | Medium              |
| 6  | High               | High      | Medium        | High             | High              | Low               | High                |
| 7  | Normal             | Low       | Low           | Low              | Low               | Medium            | Low                 |
| 8  | Very_High          | High      | High          | High             | High              | Low               | High                |
| 9  | High               | Low       | Low           | Low              | Low               | Medium            | Low                 |
| 10 | High               | Low       | Medium        | Normal           | Normal            | Medium            | Medium              |
| 11 | Normal             | Low       | Low           | Low              | Low               | Low               | Low                 |
| 12 | Very_High          | Medium    | High          | High             | High              | Low               | High                |
| 13 | High               | Low       | Medium        | Low              | Normal            | Medium            | Low                 |
| 14 | Normal             | Medium    | Medium        | Normal           | Normal            | Low               | Medium              |